Q1. (a) (i) $O(n \log n)$
(ii) $O(7^n)$
(iii) $O(n!)$
(iv) $O(n^2)$
(v) $O(\log n)$
(vi) $O(n \log n)$

(b) $\exists c=200, n_0=1 \bullet \forall n > n_0 \bullet$
$n \geq 1$
$n^3 \leq n^4$
$200n^3 \leq 200n^4$
$200n^3 \leq cn^4$.

(c) Need to prove $[\exists c, n_0 \bullet \forall n > n_0 \bullet 200n^3 \geq cn^{3.01}]$ is wrong.
Equivalent to prove for $[\forall c, n_0 \bullet \exists n > n_0 \bullet 200n^3 < cn^{3.01}]$
For any c, n_0, take $n=\max(n_0, \lceil (200/c)^{100} \rceil)+1$, we have

$n \geq (200/c)^{100}$
$n^{0.01} \geq 200/c$
$cn^{0.01} \geq 200$
$cn^{3.01} \geq 200n^3$
$200n^3 < cn^{3.01}$.

(d) $\forall c \bullet (\exists)$ take $n_0 = \max(\lceil (8^8)/(7!\ c) \rceil, 8) \bullet \forall n > n_0$

we have $(8^8)/(7!\ c) < n$	replacing n_0 by $(8^8)/(7!\ c)$
$\Rightarrow (8^8) < c(7!)n$	rewriting
$\Rightarrow (8^8)(8^{n-8}) < c(7!)n(8^{n-8})$	multiply 8^{n-8} to both sides
$\Rightarrow 8^n < c(7!)n(8)(9)\ldots(n-1)$	replacing 8^{n-8} by $(8)(9)\ldots(n-1)$
$\Rightarrow 8^n < c(1)(2)\ldots(7)(8)(9)\ldots(n-1)(n)$	rewriting
$\Rightarrow 2^{3n} < c(n!)$	rewriting

Q2. (a) (i) $T(n)=2T(n/2)+1$

By substitution.
Claim: $T(n) \leq c(n-1)$, for some $c \geq 1$, $n > 1$

$$\begin{aligned} T(n) &= 2T(n/2)+1 \\ &\leq 2\ c\ (n/2 - 1) + 1 \\ &= c\ n - 2c + 1 \\ &\leq c\ n - c \\ &= c\ (n - 1) \end{aligned}$$

By master method.
$a=2, b=2, \log_b a = 1, f(n) = 1$
$\exists\ \varepsilon = 1 > 0 \bullet$
$f(n) = 1 = n^0 = n^{(\log_2 2 - 1)} = n^{(\log_b a - \varepsilon)}$
Then, $f(n)= O(n^{(\log_b a - \varepsilon)})$
Case 1 applies, and $T(n)= \Theta(n^{(\log_b a)})=\Theta(n^1)= \Theta(n)$.

(ii) $T(n)=2T(n/2)+n$

By substitution.
Claim: $T(n) \le cn \log n$, for some $c \ge 1$, $n > 2$.
$T(n) = 2T(n/2)+n$
$\le 2\ c\ (n/2)\log(n/2) + n$
$= c\ n\ (\log n - 1)+n$
$= c\ n \log n - cn + n$
$\le c\ n \log n$

By master method
$a=2$, $b=2$, $\log_b a=1$, $f(n)= n$.
$\exists\ k = 0 \ge 0$ •
$f(n) = n = [n\text{^}(\log_2 2)] \log^0 n = [n\text{^}(\log_b a)] \log^k n$
$f(n) = \Theta(n) = \Theta(n\text{^}(\log_b a)\log^k n)$.
Case 2 applies. $T(n)= \Theta(n\text{^}(\log_b a)\log^{k+1} n)= \Theta(n\log n)$.

(iii) $T(n)=2T(n/2)+n^2$

By substitution.
Claim: $T(n) \le c\ n^2$, for some $c \ge 2$, $n > 1$
$T(n)$
$= 2T(n/2) + n^2$
$\le 2\ c\ (n/2)^2 + n^2$
$= c\ n^2/2 + n^2$
$= (c/2 + 1)\ n^2$
$\le c\ n^2$

By master method
$a=2$, $b=2$, $\log_b a=1$, $f(n)= n^2$.
$\exists \varepsilon = 1 > 0$ •
$f(n) = n^2 = n\text{^}(\log_2 2+1) = n\text{^}(\log_b a+\varepsilon)$
Then, $f(n)= \Omega(n\text{^}(\log_b a+\varepsilon))$.
Regularity condition:
$\exists\ c = 1/2 < 1$ •
$af(n/b) = 2(n/2)^2=(1/2)n^2 =(1/2)f(n) = c\ f(n)$.
Then, $af(n/b) \le cf(n)$.
Case 3 applies, and $T(n)=\Theta(n^2)$.

(b) Let p be gcd(m,n), and $m > n$.
As p is a divisor $m=k_1p$, $n= k_2p$, where k_1 and k_2 are integers which are relatively prime; otherwise p is not the greatest one.
Since $m > n$, we have $k_1 > k_2$.

Claim: k_2 and (k_1-k_2) are relatively prime.
Assume the contrary that $q = \gcd(k_2 ,k_1-k_2) > 1$.
Since q is a divisor of k_2 and (k_1-k_2), we have
$(k_1-k_2)=rq$ and $k_2 =sq$, for some positive integers r and s.
Rewriting $(k_1-k_2)=rq$, we have $k_1=k_2 + rq = (r+s)q$.
Since $k_1 = (r+s)q$, and $k_2 =sq$, $q > 1$ is then a factor of $k_1$and k_2.
$k_1$and k_2.are not relatively prime.
A contradiction.

Consider m-n.
Since n= k_2p, and m-n = (k_1-k_2)p, we have p a factor of m-n and n. Since k_2 and (k_1-k_2) are relatively prime, p is the greatest common divisor of m-n and n. Done.

Q3. (a)

```
// input is a1, a2, …, an,
// s(1..n,1..n) is an array.
for i=1 to n
  s(i,i)= ai;
  for j=i+1 to n
    s(i,j)=s(i,j-1)+ aj;
```

(b)

```
// input is a1, a2, …, an,
// s(i,j) is ready
// m(1..n,1..n) is an array.

for i=1 to n
  m(i,i)= 0;
for d=1 to n-1
  for i=1 to n-d
    j=i+d;
    min=infinity;
    for k=i to j-1
      temp = m(i,k) + m(k+1,j) + s(i,j);
      if temp < min
        min = temp
    m(i,j)=min
```

(c) $O(n^2)$ for the algorithm in (a). $O(n^3)$ for the algorithm in (b)

(d)

For all i, where $\mathbf{0<i\le n}$, $\mathbf{m(i,i)=0}$, $\mathbf{s(i,i)=a_i}$.
For $\mathbf{0<i<j}$,
$\mathbf{m(i,j)=\min_{k=i \text{ to } j-1}\{m(i,k) + m(k+1,j) + a_i + a_{i+1} + \dots + a_j\}}$.

(e)

```
// input is a1, a2, …, an,
Algorithm m(i,j), assume j>=i.
  if j=i return 0;
  // else
  sum=0;
  for k=i to j
    sum=sum+ ak;
  min=infinity;
  for k=i to j-1
    temp = m(i,k) + m(k+1,j) + sum;
    if temp < min
      min = temp;
  return min;
```

Q4. (a) Any graph with a cycle of three edges, which are of the lowest weight.

(b) Any graph with edges of unique weights.

(c) No, because one is undirected graph, and the other is directed.

(d) In comparison-based sorting, the ordering method is known, but the order of elements is unknown. In topological sort, the order of nodes are given by the directed graph, and the algorithm just finds out the given order.

(e) Construct a graph G' as follows: Use same nodes and edges as in G, and turn all weights in the edges negative. Explanation: The minimum negative total weight in G' is the negative of the maximum total weight in G.

(f) There is no observable recursive structure in Breadth First Search.

Q5. (a) Base case: h=0.
By the definition of height, the tree must be empty and it has $2^h - 1 = 0$ nodes.
For h=1, the tree has only $2^h - 1 = 1$ node.

Inductive step:
Assume that any binary tree of height h has at most $2^h - 1$ nodes, where $1 \leq h < m$.

Consider a binary tree T of height m. By the definition of height, at least one of the left and right subtrees (of the root) is m-1, and the other's height can be any number from 0 to m-1. By induction assumption, the two subtrees together have at most $2(2^{m-1} - 1)$. With the root, T has at most $2(2^{m-1} - 1)+1 = 2^m - 1$ nodes.

Result from induction.

(b) Base step: Consider h=0.
Consider a binary tree of height zero. By the definition of height, the tree must be empty. By definition, the tree is a full binary tree. Now, it has $2^h - 1 = 0$ nodes, and the base case is done.

Inductive step:
Assume that a full binary tree of height $h \geq 0$ has $2^h - 1$ nodes.

Consider a full binary tree T of height h+1. By definition, for any node x in T, the left and right subtrees of x have the same height. Consider the left subtree of the root. All the nodes inside have the same property, and therefore, the left subtree is full. Similarly, the right subtree of the root is also full. By the definition of height, one of the left and right subtrees of the root must have height h. By the definition of full binary tree, both of these two subtrees have the same height, and it is h. By inductive assumption, both subtrees have $2^h - 1$ nodes. Together with the root, there are $1+(2^h - 1)+(2^h - 1) = 2^{h+1} - 1$ nodes.

Result from induction.

(c) Yes.

(d)

Base case: h=0. A tree having $2^h - 1=0$ nodes is empty, and by definition, it is full.

Assume that a binary tree of height $h \geq 0$ has $2^h - 1$ nodes, then it is full.

Consider a binary tree T of height h+1 has $2^{h+1} - 1$ nodes.
Since the root occupies one node, there are $2^{h+1} - 2$ nodes for the root's left and right subtrees.
By the pigeon hole principle, one of these subtrees, say the left one, has at least $2^h - 1$.
By the definition of height, the left subtree has height at most h, and according to (a), it has at most $2^h - 1$ nodes.
Hence, its height is h and has exact $2^h - 1$ nodes.
The left subtree is full, by induction assumption.
Now, the right subtree also has exact $2^h - 1$ nodes.
By the definition of height, the right subtree also has height at most h. Now, its height is exactly h, otherwise, this contradicts (a).
So, the right subtree has height h, and has $2^h - 1$ nodes.
It is also full, by induction assumption.

By the definition of a full binary tree, both left and right subtrees has the property that for any node x in both subtrees, the heights of the left and right subtrees of x are equal.

Consider the root of T, both the left and right subtree have the same height h.
Then, for any node x in T, the heights of the left and right subtrees of x are equal.

By the definition of a full binary tree, T is a full binary tree.

Result from induction.